

raft-java System Architecture

Felix Berger

June 2026

1. Overview

raft-java is a from-scratch Java 17 implementation of the Raft consensus algorithm, following Ongaro & Ousterhout’s “*In Search of an Understandable Consensus Algorithm*” and the PhD dissertation “*Consensus: Bridging Theory and Practice*”.

The project implements:

- **Core Raft** (§5): leader election, log replication, safety rules
- **Cluster reconfiguration** (§6): one-server-at-a-time membership changes
- **Log compaction** (§7): independent snapshotting with chunked InstallSnapshot
- **Client interaction** (§8): no-op entry, leader redirection
- **PreVote** (§4.2.3 / §9.6): prevents partitioned servers from disrupting the cluster
- **Performance optimizations** (§10.2): parallel leader disk writes, batching, pipelining
- **Copy-on-write snapshotting** (§5.1): async background compaction

2. System Architecture

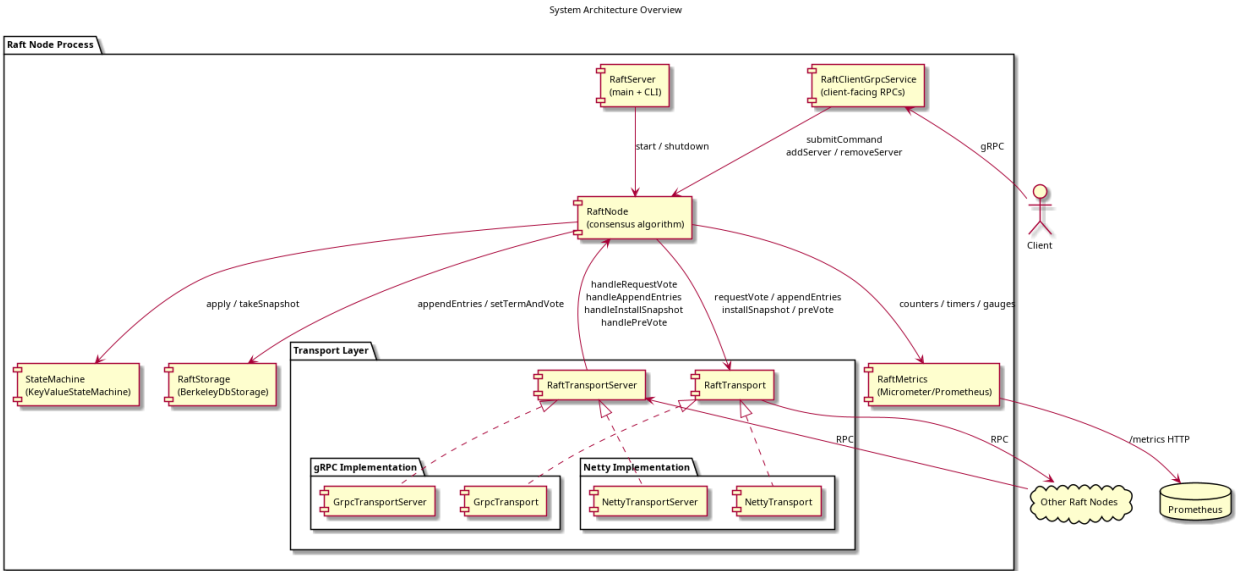


Figure 1: System Architecture Overview

Each Raft node runs as a single JVM process. The **RaftNode** class owns the consensus algorithm. It delegates durable storage to **RaftStorage** (backed by Berkeley DB JE), state machine operations to **StateMachine**, and peer communication to the **Transport** abstraction layer.

The transport layer is pluggable: a **gRPC** implementation (default) uses HTTP/2 with protobuf encoding, while a **Netty** implementation provides a lightweight TCP transport with custom length-delimited framing.

Prometheus metrics are exposed via Micrometer counters, timers, and gauges on a configurable HTTP endpoint.

3. Class Diagram

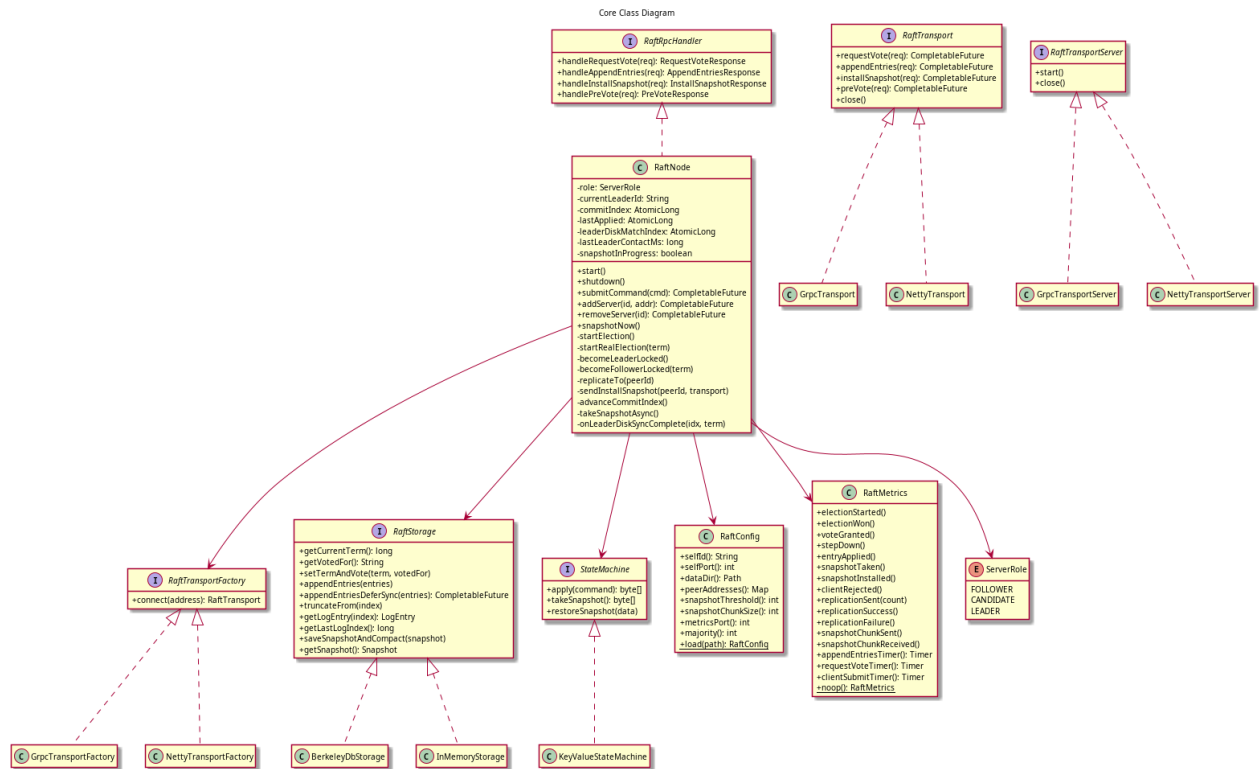


Figure 2: Core Class Relationships

Key design principles:

- **RaftNode** implements **RaftRpcHandler**, making it transport-agnostic
- All dependencies are injected (Spring IoC in production, manual wiring in tests)
- **RaftStorage** separates durable state from the algorithm
- **RaftTransport** / **RaftTransportFactory** abstract the network layer
- **RaftMetrics** instruments all significant state transitions

4. Leader Election with PreVote

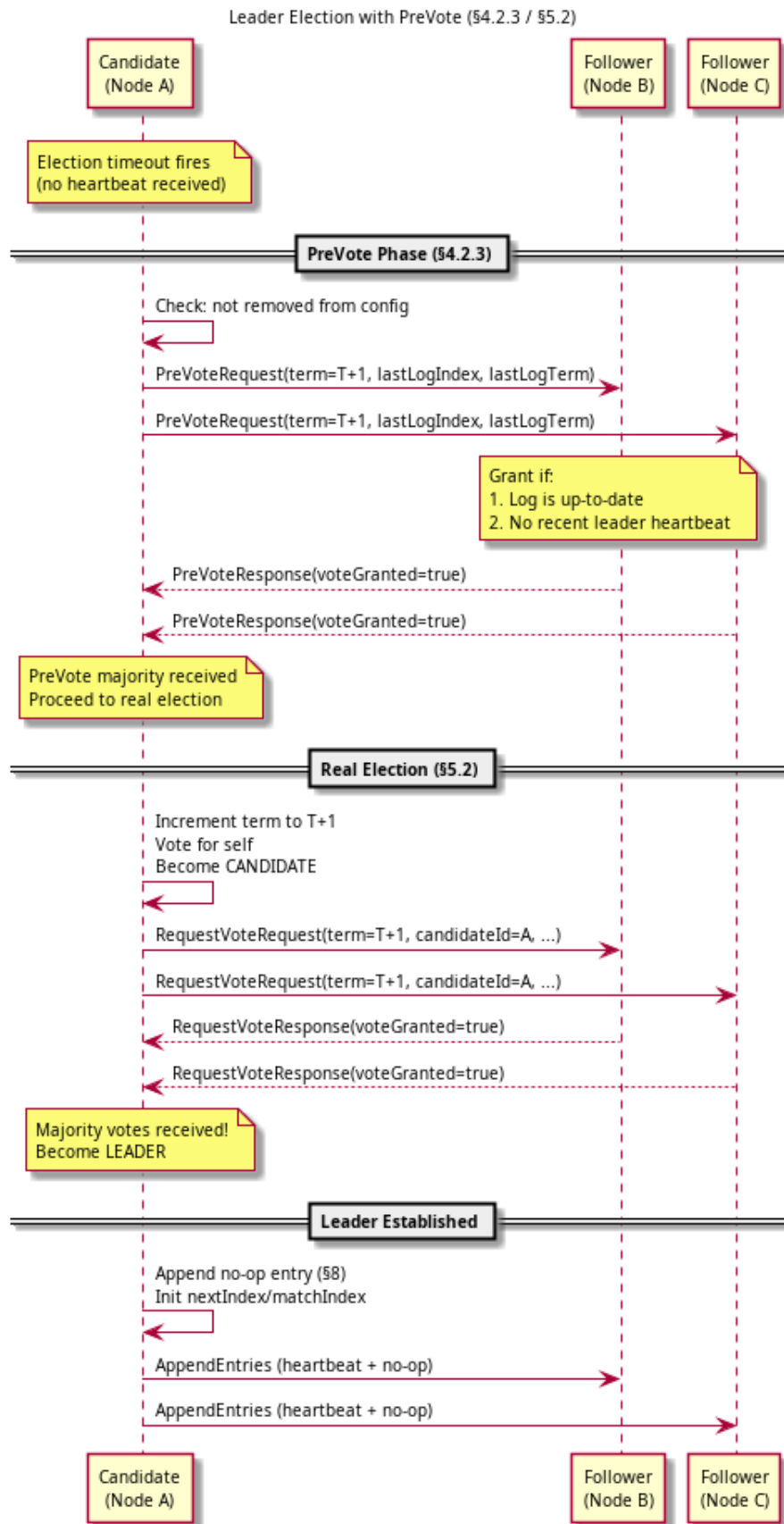


Figure 3: Leader Election Sequence

The election process has two phases:

1. **PreVote (§4.2.3)**: The candidate checks with a majority that they would grant a vote AND haven't heard from a valid leader recently. This prevents a partitioned server from incrementing its term and disrupting the cluster when it rejoins. The term is *not* incremented during PreVote.
2. **Real Election (§5.2)**: Only after PreVote succeeds does the candidate increment its term, vote for itself, and send **RequestVote** RPCs. On receiving a majority, it becomes leader and immediately appends a no-op entry (§8).

Single-node clusters skip PreVote and elect immediately.

5. Log Replication Pipeline

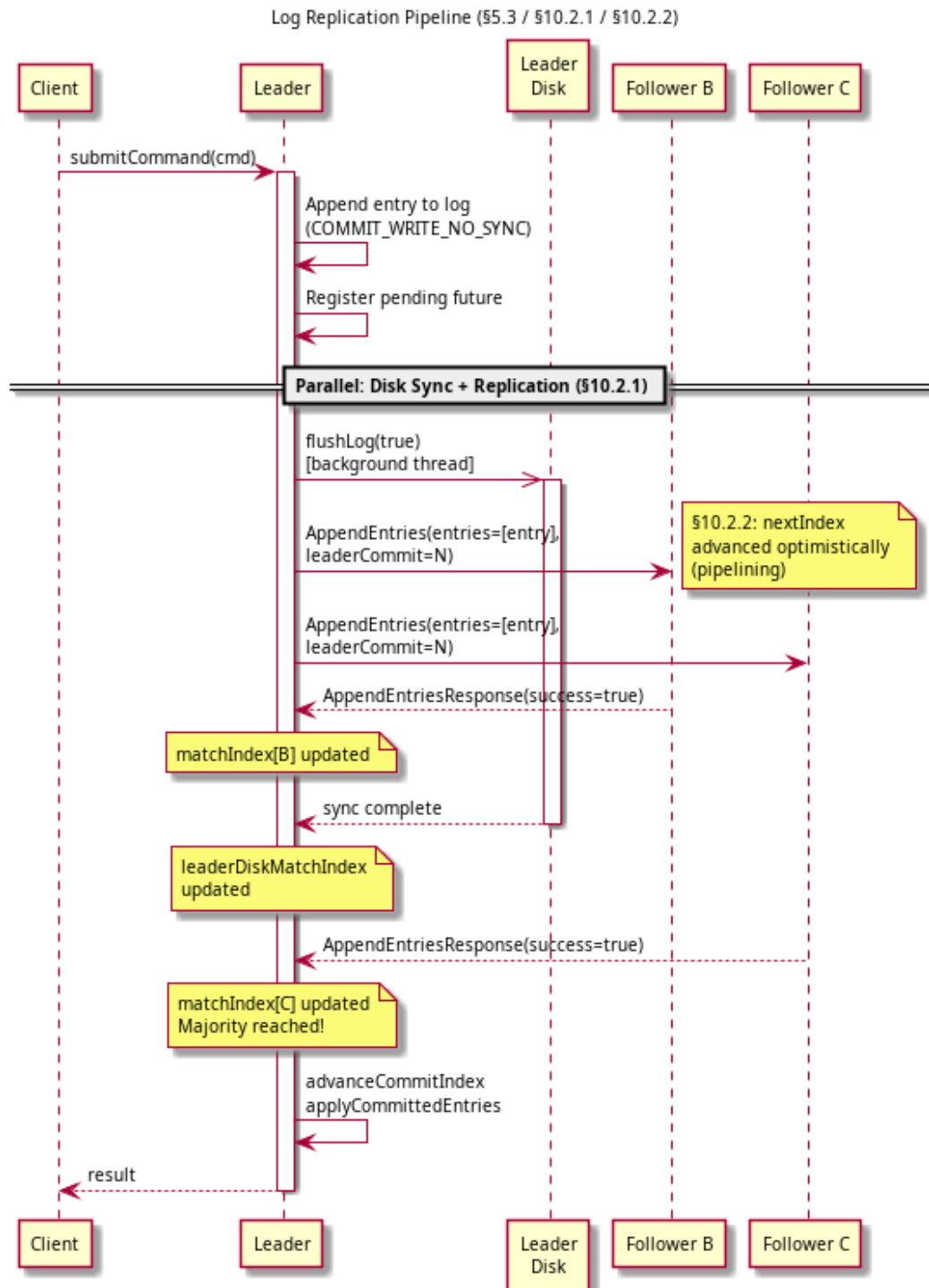


Figure 4: Replication with Parallel Disk Writes and Pipelining

Three optimizations from §10.2 reduce latency:

1. **Parallel leader disk writes (§10.2.1):** The leader writes entries with `COMMIT_WRITE_NO_SYNC` and replicates to followers immediately. A background thread calls `flushLog(true)` to fsync. The leader tracks its own durable progress via `leaderDiskMatchIndex` and only counts itself in the commit majority when the fsync is confirmed.
2. **Batching (§10.2.2):** Each `AppendEntries` RPC carries up to 1 MB of entries, amortizing per-RPC overhead under high load.
3. **Pipelining (§10.2.2):** After sending entries, the leader optimistically advances `nextIndex` so the next heartbeat tick can send subsequent entries without waiting for the previous ACK. A per-peer inflight counter (max 2) prevents unbounded pipelining. On failure, `nextIndex` reverts to the last confirmed `matchIndex + 1`.

6. Snapshot Transfer

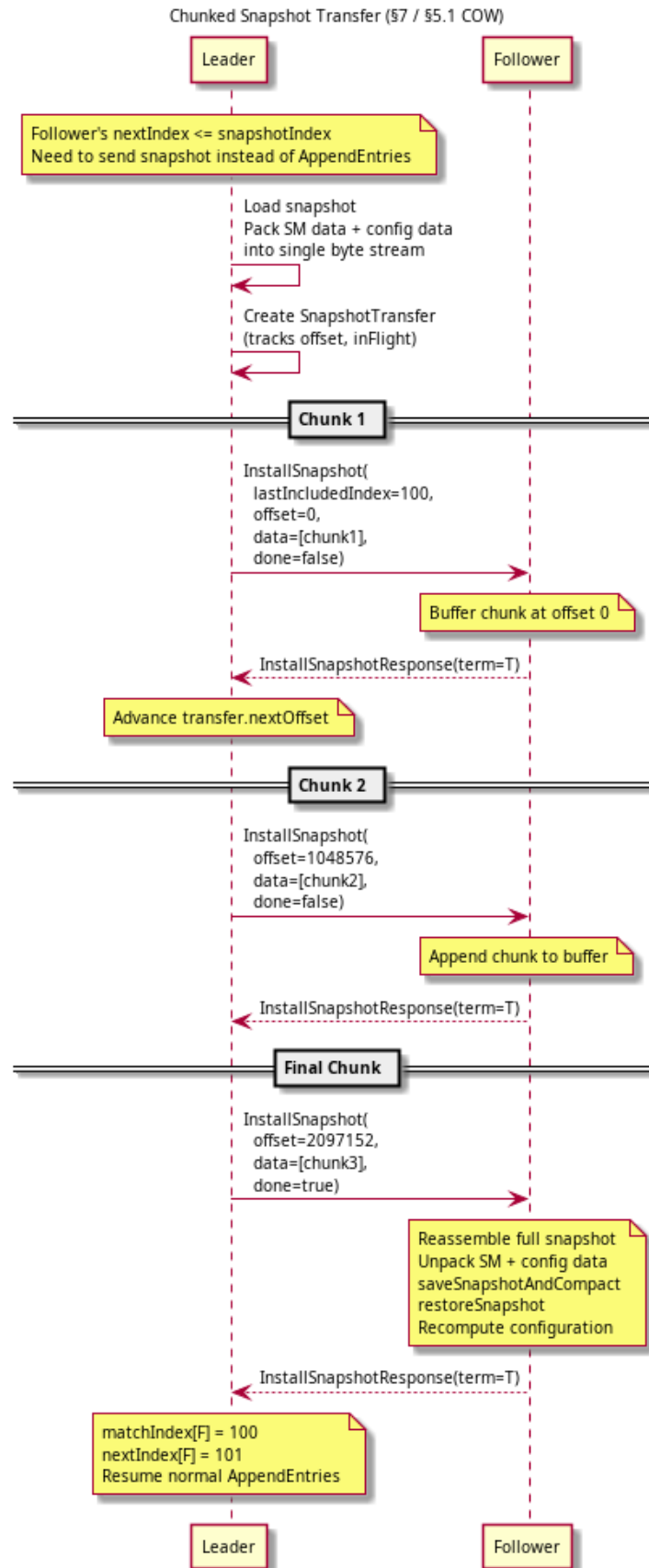


Figure 5: Chunked Snapshot Transfer

Snapshots are handled in two parts:

Taking snapshots (§5.1 COW): Automatic snapshots capture the state machine bytes under the Raft lock (fast, in-memory), then persist and compact the log on a background thread. This avoids blocking replication and elections during disk I/O. A staleness check prevents a background snapshot from overwriting a more recent `InstallSnapshot`.

Transferring snapshots (§7, Figure 13): The leader packs the state machine data and cluster configuration into a single byte stream (`[4B smLen] [smData] [cfgData]`) and sends it in fixed-size chunks (default 1 MB). One chunk is sent per heartbeat cycle. The follower buffers incoming chunks and applies the reassembled snapshot when the final chunk arrives (`done=true`).

7. Transport Layer

The transport abstraction decouples `RaftNode` from any specific network library:

Interface	Purpose
<code>RaftTransport</code>	Outbound peer connection (<code>requestVote</code> , <code>appendEntries</code> , <code>installSnapshot</code> , <code>preVote</code>)
<code>RaftTransportFactory</code>	Creates outbound connections from an address string
<code>RaftTransportServer</code>	Inbound server that routes RPCs to <code>RaftRpcHandler</code>
<code>RaftRpcHandler</code>	Handler interface for incoming RPCs (implemented by <code>RaftNode</code>)

gRPC implementation (default): Uses HTTP/2 with protobuf encoding. `GrpcTransport` wraps `RaftServiceGrpc.RaftServiceFutureStub` and converts `ListenableFuture` to `CompletableFuture`. `GrpcTransportServer` embeds a `RaftServiceAdapter` that delegates to `RaftRpcHandler`.

Netty implementation: Custom TCP protocol with length-delimited framing:

```
+-----+-----+-----+-----+
| len (4B) | type | reqId | payload |
+-----+-----+-----+-----+
```

Request IDs correlate responses to pending `CompletableFutures`. Message types 1-8 cover the four RPC pairs (`RequestVote`, `AppendEntries`, `InstallSnapshot`, `PreVote`). A shared `NioEventLoopGroup` serves all outbound connections.

8. Metrics

All significant Raft events are instrumented via Micrometer:

Metric	Type	Description
<code>raft.election.started</code>	Counter	Elections initiated
<code>raft.election.won</code>	Counter	Elections won
<code>raft.vote.granted</code>	Counter	Votes granted to candidates
<code>raft.stepdown</code>	Counter	Step-downs to follower
<code>raft.entry.applied</code>	Counter	Entries applied to state machine
<code>raft.replication.sent</code>	Counter	Entries sent to followers
<code>raft.replication.success</code>	Counter	Successful <code>AppendEntries</code> responses
<code>raft.replication.failure</code>	Counter	Failed <code>AppendEntries</code> (log mismatch)
<code>raft.snapshot.taken</code>	Counter	Snapshots taken
<code>raft.snapshot.installed</code>	Counter	Snapshots installed from leader
<code>raft.snapshot.chunk.sent</code>	Counter	Snapshot chunks sent
<code>raft.snapshot.chunk.received</code>	Counter	Snapshot chunks received
<code>raft.client.rejected</code>	Counter	Client submissions rejected (not leader)

Metric	Type	Description
<code>raft.rpc.append.entries</code>	Timer	AppendEntries RPC handling time
<code>raft.rpc.request.vote</code>	Timer	RequestVote RPC handling time
<code>raft.rpc.install.snapshot</code>	Timer	InstallSnapshot RPC handling time
<code>raft.client.submit</code>	Timer	Client submit-to-commit time
<code>raft.node.term</code>	Gauge	Current Raft term
<code>raft.node.commit.index</code>	Gauge	Highest committed log index
<code>raft.node.last.applied</code>	Gauge	Last applied log index
<code>raft.node.role</code>	Gauge	Current role (0=follower, 1=candidate, 2=leader)
<code>raft.cluster.size</code>	Gauge	Number of nodes in configuration